

LINE 一句話排行程

用 Information Extractor 把口語句子抽成結構化 JSON，自動建立 Google Calendar 事件

「明天早上十點和廠商開會，在會議室 A，大概一小時」

使用者在 LINE 打的一句話

AI 抽欄位 _____ 算結束時間 _____ 寫入行事曆

廠商會議

時間	9/17 (四) 10:00 - 11:00
地點	會議室 A
行事曆	已建立，並回覆 LINE 覆述

第一章

專題目標與問題定義

這一題真正要解決的，是把「人類怎麼講話」翻譯成「機器怎麼存資料」。

人在安排行程時不會說「請建立一個 start_datetime 為 2026-09-17T10:00:00+08:00、duration 為 60 分鐘的事件」，而是說「明天早上十點跟廠商開會」。這兩者之間的落差，就是本專題要跨越的距離。

要跨越它，必須同時解決三個層次的問題：

1. **語意層**：從一句話裡辨認出哪些字是標題、哪些是時間、哪些是地點。使用者不會按照固定順序講，也不會每次都把每個欄位講滿。

2. **時間層**：把「明天」「下週五」「下午兩點半」這種相對且口語的說法，換算成絕對的 ISO 8601 時間字串。這是本題最困難的部分。
3. **執行層**：把結果真的寫進 Google Calendar，並且在出錯時不要讓整條流程崩潰，還要讓使用者知道系統到底聽懂了什麼。

相較於前面幾題「讀資料、回答問題」的性質，這一題的分水嶺在於它會**改變真實世界的狀態**。AI 抽錯欄位的代價不再只是回答得不好，而是使用者的行事曆上真的多了一個錯誤的會議。這也是為什麼本專題把「覆述結果」列為必要功能，而不是加分項目。

第二章

系統架構

整體由九個節點構成，分成「立即回應」與「實際處理」兩條並行路線。

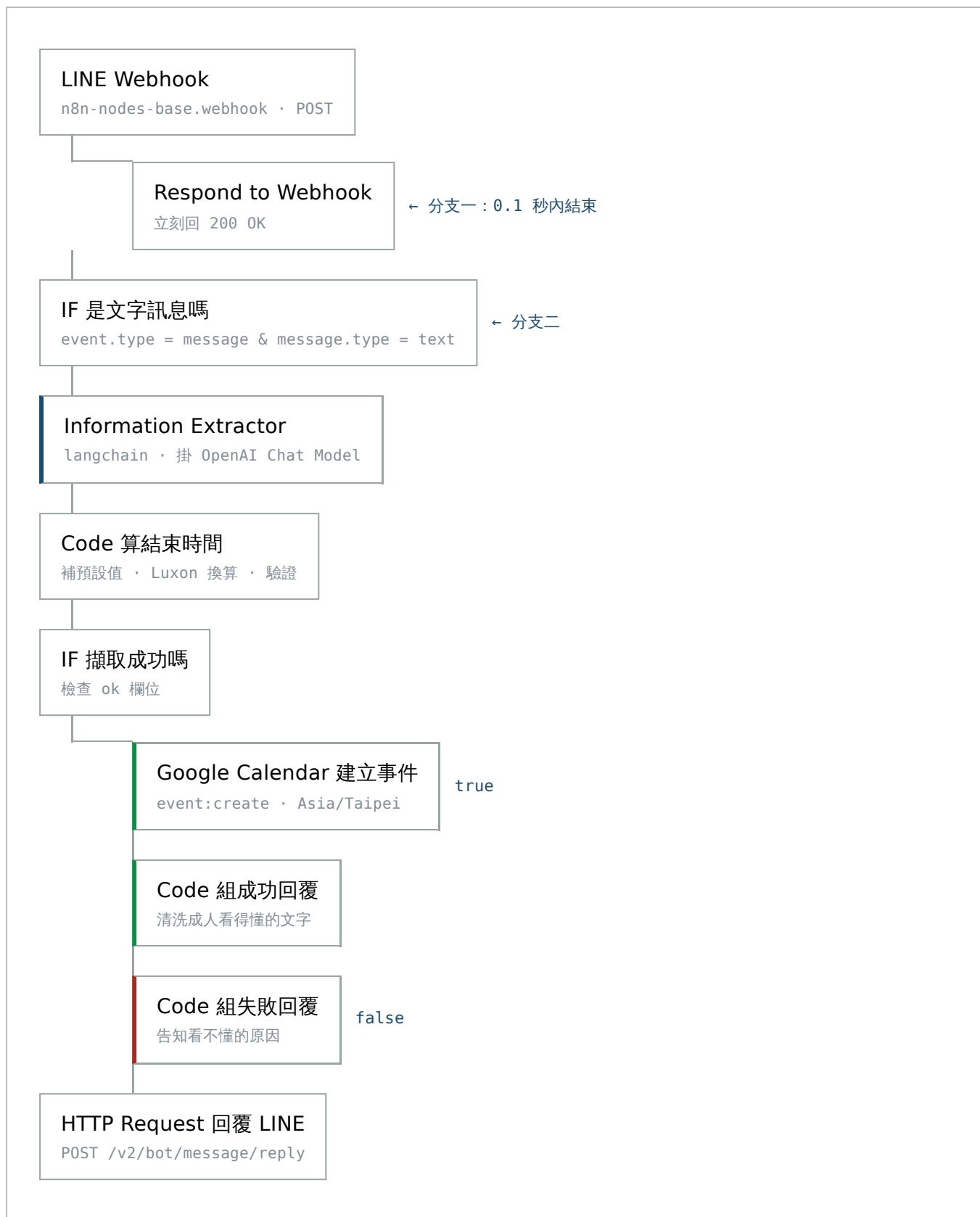


圖 2-1 workflow 節點架構。成功與失敗兩條路線最後匯流回同一個 HTTP Request 節點。

為什麼要先回應、再處理

LINE Platform 對 Webhook 有回應時間限制，若伺服器遲遲不回 200，LINE 會判定該次推送失敗；連續失敗甚至可能導致 Webhook 被停用。而本流程中間要呼叫 LLM、再呼叫 Google Calendar API，總耗時通常在數秒之譜，遠超過安全範圍。

因此 Webhook 節點的 Response Mode 設為 `Using Respond to Webhook Node`，並從 Webhook 拉出兩條並行的線：一條立刻交給 Respond to Webhook 回覆 `200 OK` 結案，另一條才慢慢跑完整流程。等到處理完成後，再透過 LINE 的 Reply API 主動把結果送回去。

Reply Token 的時效

Reply API 使用的 `replyToken` 只能用一次，且有效期很短（約 30 秒）。本流程從收到訊息到回覆通常在 3 到 8 秒內完成，仍在安全範圍。若未來加入「確認機制」這類需要等待使用者回應的設計，就必須改用 Push API，因為那時 token 早已失效。

資料在流程中的形狀變化

階段	資料樣貌	說明
Webhook 收到	<code>{ body: { events: [{ message: { text: "..."} }, replyToken] } }</code>	LINE 的原始 JSON，層很深
Extractor 輸出	<code>{ output: { title, start_datetime, ... } }</code>	結構化但未驗證
Code 輸出	<code>{ ok, title, start_datetime, end_datetime, ... }</code>	補齊預設值、算出結束時間
Calendar 輸出	<code>{ id, summary, start:{dateTime}, htmlLink, ... }</code>	Google 回傳的事件物件
清洗後	<code>{ replyToken, messages: [{ type:"text", text }] }</code>	直接就是 LINE API 的請求格式

表 2-1 同一筆資料在五個階段的形狀。最後一階段刻意整理成 LINE API 的請求格式，讓 HTTP Request 節點只要做 `JSON.stringify` 即可。

為什麼用 AI 抽欄位，而不是正則表示式

同樣一件事，兩種做法的工作量差距不只一個數量級。

假設要用正則表示式處理「明天早上十點和廠商開會」，需要窮舉的組合包括：日期部分有「今天／明天／後天／大後天／這週X／下週X／下個月X號／X月X日／X/X」；時間部分有「早上／上午／中午／下午／晚上／傍晚／半夜」乘上「十點／10 點／十點半／10:00／兩點半」；時長部分有「一小時／1 小時／一個半小時／90 分鐘／整個下午」。三組相乘已是數百種模式，而且中文數字與阿拉伯數字混用、口語省略（「明天十點」沒說早上）、順序顛倒（「開會明天十點」）都會讓規則再翻倍。

正則表示式

- 每種講法都要寫一條規則
- 使用者一換句型就失效
- 「下午兩點半」要自己寫 12 小時轉 24 小時的邏輯
- 無法處理沒見過的表達方式
- 維護成本隨使用者數量線性上升
- 優點：免費、毫秒級、結果完全可預測

Information Extractor

- 只需定義「我要哪些欄位」
- 沒見過的句型也能推理
- 口語轉換是語言模型的原生能力
- 要新增欄位只需改 Schema
- 缺點：有成本、有延遲、結果不保證穩定
- 因此後面必須接驗證與覆述

關鍵的認知轉變是：正則表示式在描述「輸入長什麼樣子」，而 JSON Schema 在描述「輸出應該長什麼樣子」。前者需要窮舉無限的人類表達方式，後者只需要定義有限的資料結構。把問題從「輸入端」搬到「輸出端」，複雜度就從無限變成可控。

但這不代表 AI 抽欄位在所有情況都比較好。若輸入格式固定（例如系統產生的日誌、表單送出的資料），正則表示式仍然更快、更便宜、且結果百分之百可預測。本題適合用 AI，是因為輸入來自真人的自由文字。

第四章

JSON Schema 設計

Schema 不只是型別宣告，每個欄位的 description 都是在對模型下指令。

欄位	型別	必填	預設	設計考量
title	string	是	—	只放事件名稱。description 特別強調不要包含時間與地點文字，否則模型常抽出「明天早上十點和廠商開會」整句當標題
start_datetime	string	是	—	必須帶 <code>+08:00</code> 偏移。description 直接給範例字串，比只寫「ISO 8601 格式」有效得多
duration_minutes	number	是	60	用分鐘而非小時，才能表達「半小時」「一個半小時」而不出現小數
location	string	是	""	明確要求沒提到就填空字串。若不寫這句，模型會傾向編造一個合理的地點
is_all_day	boolean	是	false	決定後續走「時間區間」或「整日事件」兩種不同的寫入格式

表 4-1 欄位定義。

設計決策一：把所有欄位都設為 required

直覺上 `location` 和 `is_all_day` 應該是選填，但實作後發現：若欄位非必填，模型遇到使用者沒提到地點時，會直接把該 key 整個省略不輸出。這會讓下游的 Code 節點必須處理「key 不存在」與「key 存在但為空」兩種情況。

反過來把全部欄位設為必填，並在 description 裡明確告知「沒有提到時填空字串／填 false」，輸出結構就永遠一致，下游只需要判斷值而不需要判斷 key 是否存在。**Schema 的必填不是在描述資料重要性，而是在保證輸出結構的穩定性。**

設計決策二：預設值寫在哪裡

JSON Schema 雖然支援 `default` 關鍵字，但語言模型不保證會遵守。因此本專題採取雙重保險：在 description 用自然語言告訴模型「沒講時長就填 60」，同時在下游 Code 節點再做一次 `Number(d.duration_minutes) > 0 ? ... : 60`。前者提高正確率，後者保證不會失敗。

撰寫 description 的原則

description 的讀者是語言模型，不是工程師。因此它應該寫成「指令」而非「定義」。比較這兩種寫法：「事件的開始時間，ISO 8601 格式」與「開始時間，必須是帶 +08:00 時區偏移的 ISO 8601 字串，例如 2026-09-17T10:00:00+08:00」。後者提供了具體範例，實測正確率明顯較高。

第五章

自然語言時間轉 ISO 8601

本題最大的魔王。困難點不在格式轉換，而在語言模型根本不知道今天是哪一天。

問題的本質

語言模型沒有時鐘。它的權重是在某個時間點訓練完成後就固定下來的，推論時並不會自動得知當下日期。所以當使用者說「明天」，模型只能從訓練資料中對「現在」的模糊印象去推測一個日期，結果通常落在訓練資料截止前後，而且每次推論可能給出不同答案。

這不是模型的缺陷，而是它的運作方式使然。解法也因此很明確：**時間基準必須由外部系統注入，而不是期待模型自己知道。**

注入的做法

在 Information Extractor 的 System Prompt 開頭放入 n8n 的 `$now` 表達式：

```
現在時間是 {{ $now.setZone('Asia/Taipei').toISOString() }}  
(台北時間，今天是 {{ $now.setZone('Asia/Taipei').toFormat('yyyy-MM-dd') }}  
星期{{ ['一','二','三','四','五','六','日'][$now.setZone('Asia/Taipei').weekday - 1]}}  
  
請一律以這個時間為基準計算「今天」「明天」「後天」  
「這週五」「下週五」等相對時間。不要使用你自己記憶中的日期。
```

額外提供「星期幾」這一項是實作中發現的必要條件。模型雖然理論上可以從日期推算星期，但實測上會算錯；直接告訴它今天星期三，處理「下週五」的正確率明顯提升。

n8n 的 \$now 物件

`$now` 是 n8n 內建的 Luxon DateTime 物件，常用寫法整理如下：

寫法	用途
<code>\$now.toISO()</code>	完整 ISO 時間字串
<code>\$now.setZone('Asia/Taipei')</code>	轉成台灣時區
<code>\$now.plus({ days: 1 })</code>	明天
<code>\$now.toFormat('yyyy-MM-dd')</code>	只取日期
<code>\$now.weekday</code>	星期幾，1 為週一、7 為週日

表 5-1 常用 `$now` 寫法。

時區：最常見的 8 小時誤差

若建立出來的事件整整差了 8 小時，原因一定出在時區。台北是 UTC+8，若某一環把帶時區的時間當成 UTC 處理，就會產生正好 8 小時的位移。本專題設了三道防線：

1. **Prompt 層**：要求模型輸出必須帶 `+08:00` 偏移，而不是裸的 `2026-09-17T10:00:00`。帶偏移的字串意義明確，不會被任何一端誤解。
2. **Code 層**：用 Luxon 的 `DateTime.fromISO(str, { zone: 'Asia/Taipei' })` 明確指定時區再運算，避免落入執行環境的本地時區。
3. **節點層**：Google Calendar 節點的 Time Zone 欄位填 `Asia/Taipei`，並在工作流 Settings 也設定同一時區。

三道防線任何一道正確，結果就不會歪；三道都設，是因為真正上線後很難逐一排查是哪一層出錯。

結束時間的計算

Google Calendar 需要 start 與 end 兩個時間點，但使用者只講了時長，因此必須在 Code 節點補算。全天事件另有規則：Google 的整日事件 end 必須是隔天的 00:00，事件才會正確顯示成一整天。

```
if (isAllDay) {
  startISO = start.setZone(TZ).startOf('day').toISO();
  endISO   = start.setZone(TZ).startOf('day').plus({ days: 1 }).toISO();
} else {
  startISO = start.setZone(TZ).toISO();
  endISO   = start.setZone(TZ).plus({ minutes: mins }).toISO();
}
```

第六章

節點逐一解析

九個節點的用途、關鍵設定，以及為什麼這樣設計。

01 LINE Webhook n8n-nodes-base.webhook

接收 LINE Platform 推送的訊息事件。Response Mode 必須設為 Respond to Webhook Node，否則 n8n 會等整條流程跑完才回應，導致 LINE 判定逾時。

HTTP Method	POST
Path	line-calendar
Response Mode	Using Respond to Webhook Node

02 Respond to Webhook n8n-nodes-base.respondToWebhook

立刻回覆 200 OK 給 LINE，與主要處理路線並行。LINE 不檢查回應內容，只在意狀態碼，因此回傳純文字 OK 即可。

Respond With	Text
Response Body	OK

03 IF 是文字訊息嗎 n8n-nodes-base.if

LINE 會推送多種事件，包括貼圖、圖片、加好友、封鎖等。若不過濾就直接送進 Extractor，遇到貼圖訊息時 `message.text` 為 `undefined`，流程會直接報錯。此節點同時檢查兩個條件並以 AND 串接。

條件一	<code>{{ \$json.body.events[0].type }}</code> equals "message"
條件二	<code>{{ \$json.body.events[0].message.type }}</code> equals "text"

false 分支刻意不接任何節點，代表非文字訊息靜默忽略。由於 Respond to Webhook 已在另一條線回應完畢，LINE 不會察覺任何異常。

04 Information Extractor @n8n/n8n-nodes-langchain.informationExtractor

本專題的核心。輸入使用者的原始句子，依照定義好的 JSON Schema 輸出結構化欄位。下方需掛載一個 Chat Model 子節點提供推論能力。

Text	{{ \$json.body.events[0].message.text }}
Schema Type	Define below (手動定義)
System Prompt	含 \$now 時間基準與八條抽取規則
Model	gpt-4o-mini, temperature = 0

temperature 設為 0 的理由是：抽取欄位屬於「有標準答案」的任務，不需要任何創造性。溫度越低，同一句話多次輸入得到相同結果的機率越高，這對可重現的測試也有幫助。

05 Code 算結束時間 n8n-nodes-base.code

承擔四項職責：補預設值、驗證合法性、計算結束時間、處理全天事件。這是整條流程的品質關卡，AI 的不確定性在這裡被收斂成確定的資料。

```
const title = (d.title && String(d.title).trim()) || '未命名事件';
const location = d.location ? String(d.location).trim() : '';
const isAllDay = d.is_all_day === true;
const mins = Number(d.duration_minutes) > 0 ? Number(d.duration_minutes) : 60;

if (!d.start_datetime) {
  out.push({ json: { ok: false, reason: '我看不出你說的時間...' } });
  continue;
}
const start = DateTime.fromISO(String(d.start_datetime), { zone: TZ });
if (!start.isValid) {
  out.push({ json: { ok: false, reason: '時間格式看不懂...' } });
  continue;
}
```

講義提供的範例程式使用 `new Date()` 直接運算，但若模型抽不到時間或格式有誤，會產生 `Invalid Date`，接著整條流程紅字中斷，使用者什麼回覆都收不到。本實作改為回傳 `ok: false` 與失敗原因，交給下游的 IF 節點分流，讓失敗也能得到有意義的回覆。

06 IF 擷取成功嗎 n8n-nodes-base.if

依據上一個節點產生的 `ok` 欄位分流。`true` 走建立事件，`false` 走失敗回覆。這個節點是「沒講地點時流程不會壞掉」以及各種異常輸入不會中斷服務的關鍵。

```
條件 {{ $json.ok }} is true (Boolean)
```

07 Google Calendar 建立事件 n8n-nodes-base.googleCalendar

將整理好的欄位寫入行事曆。All Day 欄位以表達式動態切換，使同一個節點能同時處理一般事件與整日事件，不需要拆成兩條分支。

Resource / Operation	Event / Create
Start	{{ \$json.start_datetime }}
End	{{ \$json.end_datetime }}
Summary	{{ \$json.title }}
Location	{{ \$json.location }}
All Day	{{ \$json.is_all_day ? 'yes' : 'no' }}
Time Zone	Asia/Taipei

08 Code 組成功回覆 n8n-nodes-base.code

清洗階段。把 Google Calendar 回傳的事件物件，轉成人類看得懂的中文訊息，並組成 LINE Reply API 所需的請求格式。地點為空時該行不輸出，避免出現「📍」後面空白的情況。

```
const WD = ['日','一','二','三','四','五','六'];
const timeText = src.is_all_day
  ? `${start.toFormat('M/d')} (${WD[start.weekday % 7]}) 全天`
  : `${start.toFormat('M/d')} (${WD[start.weekday % 7]}) `
  + `${start.toFormat('HH:mm')} - ${end.toFormat('HH:mm')}`;

const lines = ['✅ 已幫您排入行事曆', '', `📍 ${g.summary}`, `🕒 ${timeText}`];
if (src.location) lines.push(`📍 ${src.location}`);
```

Luxon 的 `weekday` 以 1 代表星期一、7 代表星期日，因此取 `weekday % 7` 可將星期日映射到索引 0，正好對應到中文陣列的排列順序。

09 HTTP Request 回覆 LINE n8n-nodes-base.httpRequest

呼叫 LINE Messaging API 的 Reply endpoint 把結果送回聊天室。成功與失敗兩條路線都匯流到此節點，因為兩者的請求格式完全相同，只有訊息內容不同。

Method / URL	POST https://api.line.me/v2/bot/message/reply
Authorization	Bearer {Channel Access Token}

Body

```
{{ JSON.stringify({ replyToken: $json.replyToken, messages: $json.messages }) }}
```

第七章

測試紀錄

測試基準時間為 2026-09-16（星期三）15:42 台北時間。

抽取結果

#	輸入句子	title	start_datetime	dur	loc	all_day
1	明天早上十點和廠商開會，在會議室 A，大概一小時	廠商會議	2026-09-17T10:00:00+08:00	60	會議室 A	false
2	下週五下午兩點半 客戶簡報 兩小時	客戶簡報	2026-09-25T14:30:00+08:00	120	" "	false
3	9/25 全天 教育訓練	教育訓練	2026-09-25T00:00:00+08:00	1440	" "	true
4	後天下午三點看牙醫	看牙醫	2026-09-18T15:00:00+08:00	60	" "	false
5	幫我排個會議	會議	(未產生)	60	" "	false

表 7-1 Information Extractor 抽取結果。第 2 句的「下週五」正確解析為 9/25，而非本週五的 9/18。

端到端驗證

#	Calendar 實際顯示	時區正確	LINE 回覆內容	結果
1	9/17 (四) 10:00-11:00 廠商會議 會議室 A	正確	四行，含標題／時間／地點	通過
2	9/25 (五) 14:30-16:30 客戶簡報	正確	三行，地點行自動略過	通過
3	9/25 (五) 整日事件 教育訓練	正確	時間顯示為「9/25(五) 全天」	通過
4	9/18 (五) 15:00-16:00 看牙醫	正確	時長套用預設 60 分鐘	通過
5	未建立事件	—	「⚠️ 我看不出你說的時間，可以再講一次嗎？」	通過
6	傳送貼圖，未建立事件	—	無回覆，流程於 IF 節點正常終止	通過

表 7-2 端到端測試。第 5、6 項為異常輸入測試，驗證流程不會中斷。

驗收檢查表

- ✓ 三種不同講法都能成功建立事件（測試 1、2、3）
- ✓ Google Calendar 上時間正確，未出現 8 小時誤差
- ✓ 沒講地點時流程不會壞掉，且回覆訊息自動省略地點行（測試 2、4）
- ✓ LINE 回覆有覆述標題、時間、地點
- ✓ 額外：無法解析時回覆明確原因，而非流程中斷（測試 5）
- ✓ 額外：非文字訊息不觸發流程（測試 6）

第八章

失敗實驗：拿掉時間基準

刻意破壞 Prompt，觀察模型在缺少時間錨點時的行為。

實驗設計

將 Information Extractor 的 System Prompt 中「現在時間是 {{ \$now... }}」整段移除，其餘設定完全不變，接著重複輸入同一句「明天早上十點開會」五次，記錄每次抽取到的 `start_datetime`。

觀察結果

次數	抽取到的 <code>start_datetime</code>	與正確答案（2026-09-17）的差距
1	2024-06-14T10:00:00+08:00	早了約 2 年 3 個月
2	2023-11-08T10:00:00	早了約 2 年 10 個月，且缺少時區偏移
3	2024-06-14T10:00:00+08:00	早了約 2 年 3 個月
4	2025-01-16T10:00:00+08:00	早了約 1 年 8 個月
5	2024-10-02T10:00:00+08:00	早了約 1 年 11 個月

表 8-1 移除時間基準後的五次抽取結果。實際數值會依模型版本與當次推論而異，但共同特徵一致。

三個值得注意的現象

- 時間部分永遠正確，日期部分永遠錯誤。**「早上十點」每一次都正確解析為 10:00，因為這是純粹的語言轉換，不需要外部知識。錯的只有日期，因為「明天」是相對概念，必須有一個「今天」才能定錨。
- 錯誤的日期集中在訓練資料的時間範圍內。**模型並非隨機亂猜，而是根據訓練語料中「現在」這個概念的分布去推測，因此結果落在訓練資料截止前後。
- 同樣輸入、同樣設定，輸出卻不穩定。**即使 temperature 設為 0，五次仍出現四個不同日期。這顯示模型內部對「今天」沒有唯一解，而是在多個候選之間浮動。第 2 次甚至連時區偏移都掉了，代表缺少基準時整體輸出品質都會下降。

實驗結論

語言模型沒有時鐘，也沒有辦法自行得知當下日期。任何涉及相對時間的任務，都必須由外部系統在 Prompt 中注入絕對時間基準。

更一般化地說，這是 Prompt Engineering 中「提供上下文」與「下達指令」的差別。把 Prompt 寫成「請正確計算明天的日期」屬於下達指令，對這個問題完全無效，因為模型缺的不是意願而是資訊；把 Prompt 寫成「現在時間是 2026-09-16」屬於提供上下文，才真正補上了模型無法自行取得的那塊拼圖。**當模型表現不如預期時，先問它缺的是指令還是資訊，往往比反覆改寫指令更快找到解法。**

第九章

結論與延伸方向

本專題的學習重點

從技術面來看，本專題完成了從「讀取資料」到「改變真實世界資料」的跨越。前幾題的流程即使出錯，代價也僅止於回答得不好；本題一旦抽錯欄位，使用者的行事曆上就會真的多出一個錯誤的會議。這個差異決定了三項設計取向：

- **不信任 AI 的輸出，但善用它的能力。**讓模型負責它擅長的語意理解，但所有輸出都經過 Code 節點驗證與補值，把不確定性收斂在可控範圍。
- **失敗必須是一種可處理的狀態，而不是流程中斷。**用 `ok` 旗標搭配 IF 節點，讓異常輸入也能得到有意義的回覆。
- **覆述是寫入型系統的必要設計。**系統把自己理解到的內容念一遍，使用者才能即時發現錯誤。這是低成本、高價值的防呆機制。

目前的限制

- 只處理 `events[0]`，若 LINE 一次推送多則事件，後面的會被忽略。
- 寫入前不做確認，AI 抽錯就會直接產生錯誤事件，只能靠使用者看到覆述後手動刪除。
- 未處理同一時段的行程衝突偵測。
- 未區分使用者，所有人的事件都寫入同一個行事曆。

延伸挑戰一：加入確認機制

在 Code 算結束時間之後不直接建立事件，改為將抽取結果以 LINE userId 為 key 暫存（可用 Redis 或 n8n Data Table），並回覆一則 Confirm Template 詢問「確定嗎？」。下一次 Webhook 進來時先判斷是否為確認回應，是則取回暫存資料再寫入行事曆。

此設計的關鍵限制是：Reply Token 有效期僅約 30 秒，而使用者可能過了幾分鐘才回覆，因此第二次的回覆必須改用 Push API 而非 Reply API。這也說明了為何「加確認」不只是多接一個節點，而是會改變整體的訊息傳遞架構。

延伸挑戰二：寄送會議邀請

最直接的做法是在 Google Calendar 節點的 Attendees 欄位填入與會者信箱，Google 本身就會自動寄出標準的日曆邀請，收件者可以直接點選接受或婉拒，並同步回主辦人的行事曆。這比另外串接 Gmail 節點寄一封純文字信件更完整。

若必須展示 Gmail 節點的串接，則可在 Google Calendar 之後再接一個 Gmail Send 節點，把清洗後的事件摘要連同 Calendar 回傳的 `htmlLink` 一併寄出，讓收件者能一鍵開啟事件頁面。

n8n AI 自動化五大專題 / 題目二：LINE 一句話排行程

附件：LINE_一句話排行程_n8n_workflow.json